

Documentación de Proyecto: Organizador de Coches

Este documento detalla el funcionamiento, la arquitectura y el despliegue de la aplicación web diseñada para organizar el transporte en coches para eventos de fin de semana.

1. Resumen de Arquitectura y Despliegue

La aplicación ha sido desarrollada pensando en la simplicidad y en el ecosistema móvil, para que los usuarios puedan interactuar fácilmente desde enlaces en WhatsApp u otras aplicaciones de mensajería.

Componente	Tecnología / Ubicación
Framework Web	Python con Streamlit
Repositorio (Código Fuente)	GitHub (Proyectos-Personales-Diego/Organizar-Co ches)
Plataforma de Despliegue	Streamlit Community Cloud
URL de Producción	https://coches-garfio.streamlit.app

2. Manual de Usuario (Flujo de la App)

La interfaz consta de tres flujos principales que permiten a cualquier miembro del grupo autogestionarse sin depender de un administrador:

- Creación de un Coche:** Los conductores despliegan el panel "Ofrecer mi coche", introducen su nombre y el número de plazas libres reales (sin contar al propio conductor). Al enviar el formulario, el vehículo aparece listado públicamente en la vista principal.
- Asignación de Plazas (Subirse):** Debajo de cada coche listado que tenga huecos libres, aparece un campo de texto. El pasajero introduce su nombre y pulsa "Me subo". El

sistema actualiza las plazas restantes automáticamente y bloquea inscripciones duplicadas si detecta el mismo nombre.

3. **Eliminación (Cancelar un coche):** Cada bloque de vehículo cuenta con un botón para borrarlo en caso de que el conductor no pueda asistir, se suspenda su viaje o haya cometido un error al crearlo.

3. Detalles Técnicos y Gestión del Estado

Al no contar con una base de datos externa tradicional para maximizar la velocidad de desarrollo y despliegue, la aplicación hace uso de la memoria de sesión del servidor web. La base de datos temporal (`st.session_state.coches`) almacena la información estructurada mediante diccionarios, siguiendo el siguiente esquema de datos en tiempo de ejecución:

```
{
  "Nombre del Conductor": {
    "plazas": 4,
    "pasajeros": ["Pasajero 1", "Pasajero 2"]
  }
}
```

Limitaciones de Arquitectura Actuales: Al desplegar la aplicación en Streamlit Community Cloud (un entorno serverless), el estado almacenado en memoria se reiniciará si la aplicación entra en reposo por inactividad (sleep mode) tras varias horas o si los contenedores se reinician. Para un uso de fin de semana, en el que la coordinación ocurre en un margen de 24-48 horas, es más que suficiente. Para un uso persistente a largo plazo, se debería sustituir el `st.session_state` por lecturas/escrituras a una pequeña base de datos externa como Supabase (PostgreSQL), Firebase o Google Sheets.

4. Archivos del Proyecto

- **app.py:** Archivo principal que contiene toda la lógica de presentación (UI) usando los widgets de Streamlit, la gestión de eventos de los formularios y la visualización del estado actual de los coches.
- **requirements.txt:** Archivo de dependencias necesario para que el entorno remoto de Streamlit sepa qué librerías de Python debe instalar. Actualmente solo requiere streamlit.